

String Cleaning, Dates, and a Clean DataFrame

Python 101 • Day 6 • pandas data cleaning workflow

Today's move

from messy
columns



to routed output

clean.csv +
rejected.csv

BLOCK 44

What this block covers

30 minutes instruction + 30 minutes guided practice

String cleanup

Use .str methods across entire columns.

Allowed values

Use isin() to detect unsupported values.

Date parsing

Convert text dates and detect invalid dates.

Clean outputs

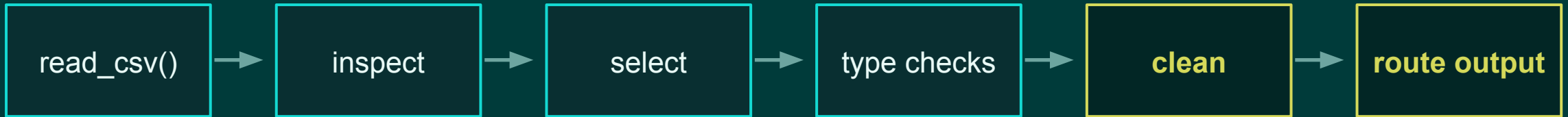
Create clean and rejected dataframes, then write CSV files.

Guiding question: How do we turn messy source data into explainable clean output?

BLOCK 44

Where we are in the morning workflow

Blocks 41–43 inspected, selected, filtered, and diagnosed quality problems.



This block turns diagnosis into a repeatable cleaning workflow.

Messy data before cleaning

Focus on problems humans can see, then express them in pandas.

record_id	source	status	count	date	errors
eq-1001	Maint	ACTIVE	12000	2026-07-01	0
EQ-1002	LOGISTICS	active	8500	not-a-date	2
	finance	inactive	bad	2026-13-40	0
eq-1004	unknown	retired	0	2026-07-04	-1

Detect

Whitespace
Capitalization
Invalid values
Bad numbers
Bad dates

The goal is not cosmetic cleanup. The goal is reliable analysis.

String operations across a column

The `.str` accessor is the doorway to string methods on a Series.

Earlier: one string

```
status = status.strip().lower()
```

Now: one column

```
records["status"] = (  
    records["status"]  
    .str.strip()  
    .str.lower()  
)
```

Same mental model, larger target: one method applied across many values.

Normalize several text fields

Clean identifiers, sources, and statuses before validation.

```
records["record_id"] = (  
    records["record_id"]  
    .str.strip()  
    .str.upper()  
)  
  
records["source"] = (  
    records["source"]  
    .str.strip()  
    .str.lower()  
)  
  
records["status"] = (  
    records["status"]  
    .str.strip()  
    .str.lower()  
)
```

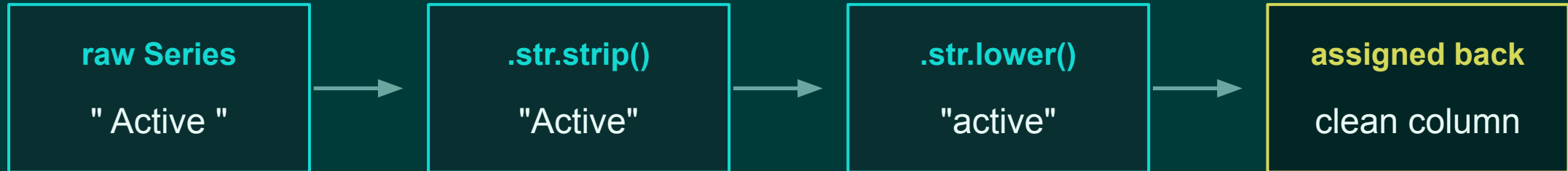
Before validating

Make equivalent values look equivalent.

before	after
Active	active
eq-1001	EQ-1001

Read chained cleanup from top to bottom

Chaining is just several steps written together.



Do not memorize the chain. Read it as a small pipeline.

Allowed-value checks with `isin()`

After normalization, compare values to the approved list.

```
allowed_statuses = [
    "active",
    "inactive",
]

invalid_status = (
    ~records["status"].isin(
        allowed_statuses
    )
)
```

Read it aloud

status is in allowed_statuses



~ flips the answer

not allowed?

Validate source and status columns

Use separate masks so the quality report can count each problem.

```
allowed_sources = [  
    "maintenance",  
    "logistics",  
    "finance",  
]  
  
invalid_source = (  
    ~records["source"].isin(  
        allowed_sources  
    )  
)  
  
invalid_status = (  
    ~records["status"].isin(  
        allowed_statuses  
    )  
)
```

Why separate masks?

Unsupported source and
unsupported status are
different data-quality findings.

Different finding → different
count → different explanation.

Numeric conversion review

Convert first, then validate numeric ranges.

```
numeric_columns = [  
    "record_count",  
    "completion",  
    "error_count",  
]  
  
for column in numeric_columns:  
    records[column] = pd.to_numeric(  
        records[column],  
        errors="coerce"  
    )
```

errors="coerce"

Attempt conversion. If conversion fails, make the value missing.

"bad"	→	NaN
"120"	→	120

Dates are data types too

Text dates must be parsed before reliable date analysis.

```
records["processed_date"] = (  
    pd.to_datetime(  
        records["processed_date"],  
        errors="coerce"  
    )  
)
```

Conversion pattern

raw text	parsed value
2026-07-01	valid date
not-a-date	NaT
2026-13-40	NaT

NaT means missing date/time.

Identifying invalid dates

After conversion, invalid dates become missing values.

```
invalid_date = (  
    records["processed_date"].isna()  
)  
  
invalid_date_count = (  
    invalid_date.sum()  
)
```

Important nuance

A missing parsed date could mean the original value was blank or invalid text.

Inspect rows before deciding how to handle them.

Conversion creates evidence. It does not automatically make the decision.

BLOCK 44

Extract basic date components

Once parsed, dates support date-oriented operations.

```
first_date = records[
    "processed_date"
].min()

last_date = records[
    "processed_date"
].max()

records["processed_month"] = (
    records["processed_date"].dt.month
)
```

Keep it light

This is not a time-series lesson yet. The goal is to show why date types matter.

Typed values unlock better questions.

Create a rejection reason column

A clean/rejected split needs an explanation for rejected rows.

```
records["rejection_reason"] = ""
```

Mental model

reason	route
""	clean
message	rejected

One reason per row is enough for this introductory workflow.

Blank reason means no rejection rule has flagged the row.

Assign reasons with .loc

Use a mask to choose rows and a column name to choose where to write.

```
records.loc[
    records["record_id"].isna(),
    "rejection_reason"
] = "missing record ID"

records.loc[
    records["record_count"].isna(),
    "rejection_reason"
] = "invalid record count"
```

.loc reads like

For rows where this mask is True,
write this value into this column.

```
rows, column = value
```

Common rejection rules

Each rule becomes a mask and a readable reason.

Missing ID

record_id is missing

Bad range

completion outside 0–100

Unsupported value

source or status not allowed

Bad number

numeric conversion produced NaN

Negative errors

error_count below 0

Bad date

processed_date is missing after parse

Good rejected output answers: why did this row not make it into the clean dataset?

Range checks after conversion

Type conversion makes numeric rules meaningful.

```
bad_record_count = (  
    records["record_count"] <= 0  
)  
  
bad_completion = (  
    (records["completion"] < 0)  
    | (records["completion"] > 100)  
)  
  
bad_error_count = (  
    records["error_count"] < 0  
)
```

Use parentheses

Combined conditions are easier to read when each comparison is grouped.

| means OR for pandas masks.

Rule order affects the first reason saved

Intro workflow: one rejection reason per row.

Simple strategy

Run the most basic rules first:

1. Missing identifiers
2. Invalid conversions
3. Invalid ranges
4. Unsupported categories

Later improvement

Advanced pipelines may accumulate multiple reasons per row.

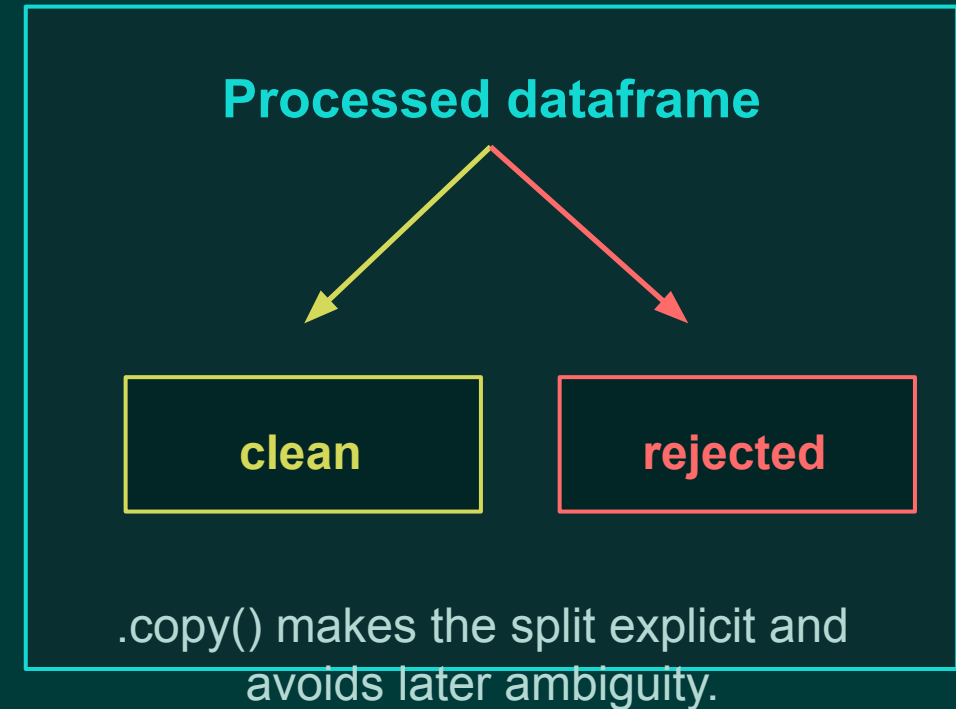
For today, one clear reason is enough.

Separate clean and rejected records

Route rows based on whether a rejection reason exists.

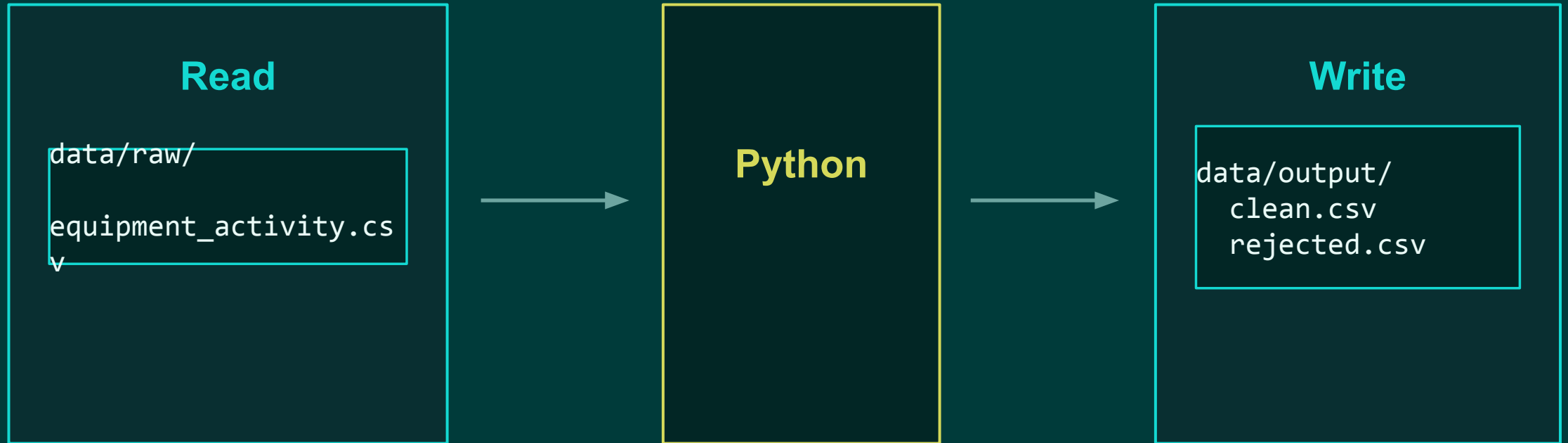
```
rejected_records = records[
    records["rejection_reason"] != ""
].copy()

clean_records = records[
    records["rejection_reason"] == ""
].copy()
```



Preserve raw input

Write new files instead of overwriting the source file.



Raw files are evidence. Output files are products.

Writing cleaned CSV output

Use index=False so pandas does not write the internal index as a column.

```
clean_records.to_csv(
    "data/output/clean_equipment_activity.csv",
    index=False
)

rejected_records.to_csv(
    "data/output/rejected_equipment_activity.csv",
    index=False
)
```

Always verify

After writing, load the files again or open them to confirm the output looks right.

```
pd.read_csv(output_file)
```

BLOCK 44

Morning cleaning workflow

This is the reusable pattern students should leave with.

`read_csv()`

`inspect`

`select columns`

`normalize text`

`convert values`

`build masks`

`assign reasons`

`split dataframes`

`to_csv()`

Cleaning is a workflow, not a single pandas command.

BLOCK 44

Exercise 1: normalize text fields

Goal: make equivalent text values look equivalent.

```
records["record_id"] = (  
    records["record_id"]  
    .str.strip()  
    .str.upper()  
)  
  
# Repeat for source and status
```

Task

Normalize record_id, source, and status. Then inspect the first ten rows.

Timebox: 5 minutes

BLOCK 44

Exercise 2: convert numbers and dates

Goal: convert raw text into usable types and count failed conversions.

```
for column in [
    "record_count",
    "completion",
    "error_count",
]:
    records[column] = pd.to_numeric(
        records[column],
        errors="coerce"
    )

records["processed_date"] = pd.to_datetime(
    records["processed_date"],
    errors="coerce"
)
```

Task

Count missing values after conversion for each converted column.

```
records.isna().sum()
```

Timebox: 6 minutes

BLOCK 44

Exercise 3: validate allowed values

Goal: use `isin()` to identify unsupported source and status values.

```
allowed_sources = [
    "maintenance",
    "logistics",
    "finance",
]

allowed_statuses = [
    "active",
    "inactive",
]

invalid_source = (
    ~records["source"].isin(allowed_sources)
)

invalid_status = (
    ~records["status"].isin(allowed_statuses)
)
```

Task

Print counts and inspect the rows that failed each rule.

```
records[invalid_source]
```

Timebox: 5 minutes

BLOCK 44

Exercise 4: assign rejection reasons

Goal: create a reason column and fill it with clear explanations.

```
records["rejection_reason"] = ""

records.loc[
    records["record_id"].isna(),
    "rejection_reason"
] = "missing record ID"

records.loc[
    invalid_source,
    "rejection_reason"
] = "unsupported source"
```

Task

Add reasons for invalid numbers, invalid ranges, unsupported status, and invalid dates.

Timebox: 7 minutes

BLOCK 44

Exercise 5: split clean and rejected rows

Goal: route rows based on the rejection_reason column.

```
rejected_records = records[
    records["rejection_reason"] != ""
].copy()

clean_records = records[
    records["rejection_reason"] == ""
].copy()

print(len(clean_records))
print(len(rejected_records))
```

Task

Inspect both dataframes. Confirm every rejected row has a readable reason.

Timebox: 4 minutes

BLOCK 44

Exercise 6: write and verify output

Goal: preserve raw data and create output products.

```
clean_records.to_csv(  
    "data/output/clean_records.csv",  
    index=False  
)  
  
rejected_records.to_csv(  
    "data/output/rejected_records.csv",  
    index=False  
)  
  
check_clean = pd.read_csv(  
    "data/output/clean_records.csv"  
)
```

Task

Write both files. Then load at least one output file again to verify it.

Timebox: 3 minutes

Applied challenge: clean an operational dataset

Use the full workflow on a dataset you did not hand-design.

Required workflow

1. Load and inspect
2. Normalize text fields
3. Convert numeric and date fields
4. Validate ranges and allowed values
5. Assign rejection reasons
6. Split clean and rejected rows
7. Write output CSV files

Success criteria

Students can answer:

What was wrong?
How did you detect it?
Did you correct or reject it?
How many rows were affected?

Success: clean output + rejected output + explainable decisions.

Block 44 wrap-up

You can now build a clean dataframe from messy source data.

Normalize

Use `.str.strip()`, `.str.lower()`,
and `.str.upper()`.

Convert

Use numeric and date
conversion with
`errors="coerce"`.

Route

Use rejection reasons to
split clean and rejected
rows.

A cleaned dataframe is not just neater. It is structured evidence that the pipeline made intentional decisions.

Next: group rows and calculate summaries with pandas.